JavaScript 1.11

Async Coordination Patterns

Print-first interview refresher | select → admit → attempt → settle

Ask two questions:
which quantity is bounded?
who owns the next attempt?

1 | Select the mechanism by the quantity you control

Debounce	Throttle	Concurrency	Retry
wait for quiet	updates during a stream	unfinished operations	eligible new attempts

These are separate contracts: frequency ≠ active count ≠ queue size ≠ retry budget.

Use one stable wrapper per owner. A timer does not move CPU work to another thread.

2 | Compare timelines before writing the wrapper

Inputs: A at start; B after 40 ms; C after 80 ms; D after 140 ms.

Wait: 100 ms. All times start at the first input; timers do not run late in this example.

Debounce: after the pause	D after 240 ms
Debounce: first call + after the pause	A at start; D after 240 ms
Our throttle: first call + end of window	A at start; C after 100 ms; D after 200 ms

One input with both options runs once. cancel drops the waiting call; flush runs it now.

3 | Queue factories; reserve before running user code

FIFO pending	→	Active slot	→	Settled
not started yet		reserve → call factory		release exactly once

Queued abort / close → remove entry + listener; reject its promise with the exact reason.

Active abort → signal the API; keep the slot until actual settlement, even if abort is ignored.

allSettled reports input order, not completion order. close does not cancel active work.

Bound the queue too. Awaiting the same last occupied slot from inside itself can deadlock.

4 | Backoff sets a cap; full jitter spreads clients inside it

```
cap = min(maxDelay, baseDelay * 2 ** (failedAttempt - 1))
```

delay = floor(random * cap), random ∈ [0, 1); zero is allowed. First failure has number 1.

Example: base=100, max=250, random=0.5 → 50, 100, 125, 125. No wait after the last attempt.

5 | Eligibility and shared budgets come before another attempt

Safe operation?	Transient failure?	Budget remains?
server idempotency contract	explicit retry policy	attempts + total time

A lost response does not prove no server effect. Cancellation is not a reason to retry.

Retry-After: seconds or HTTP-date. Three layers × three attempts → up to 27 lower calls.

6 | Propagate cancellation through every waiting layer

Owner signal → queued entry → active operation → retry sleep → no next attempt.

New search input → version++ + abort now → debounce → request → verify version → UI.

limit(retry(...)): slot includes backoff. retry(limit(...)): each attempt re-enters admission.

Dispose removes listeners, cancels pending timers and signals active work. Server effects stay separate.

Test contracts with controlled clocks and promises; verify native APIs separately.

Check edges, reentry, reverse completion, partial failure, ignored abort and close-before-response.